第23课 高精度加法

一、课程信息

项目	内容
课次	第 23 课 / 共 40 课
主题	高精度加法
前置知识	数组、循环、模拟法
课时	100 分钟

教学目标

1. **理解高精度的必要性**：能说出 `int` / `long long` 的存储上限，并举例说明哪些场景需要高精度。
2. **掌握高精度加法核心流程**：能独立写出"字符串读入 倒序存数组 逐位相加处理进位 去前导零倒序输出"的完整代码。
3. **通过课堂练习验证**：能在课堂上独立完成至少 1 道高精度加法题并通过评测。

二、教学重点与难点

重点

- 用数组模拟竖式加法的整体思路（拆位 逐位加 进位 输出）。
- 字符串转数组时的**倒序存储**技巧。
- 逐位相加时**进位的处理**。

难点

- 为什么要倒序存储（个位放 `a[0]`），而不是正序存储。
- 最高位产生进位后，结果位数比两个加数都多一位的情况。
- 输出时去掉前导零、再倒序输出的细节。

三、课堂时间分配（共 100 分钟）

时间	环节	内容	形式
0′–5′	热身导入	"超级大数字"趣味挑战，引出问题	互动提问
5′–20′	讲解①	为什么需要高精度 + 核心思路 + 倒序存储	讲授 + 板书/演示（15 分钟）
20′–30′	动手①	小练习：手动模拟竖式，把过程填进表格	纸笔练习
30′–50′	讲解②	完整代码逐行拆解（例题 1 + 例题 2）	讲授 + 现场敲码（20 分钟）
50′–55′	休息/小游戏	"进位接龙"小游戏放松	互动
55′–80′	动手②	课堂练习题，独立编码 + 提交评测	上机实践（25 分钟）
80′–90′	讲评答疑	公布练习结果，讲解常见错误	讨论
90′–97′	总结巩固	回顾核心流程，学生复述	互动
97′–100′	布置作业	课后作业 + 下节课预告	讲授

> 讲解段：讲解①（15 分钟）+ 讲解②（20 分钟），均未超过 25 分钟。

四、讲稿提纲

4.1 热身导入（0′–5′）

> 同学们，老师今天出一道数学题：请你用 C++ 算一下——
> **99999999999999999999 + 1 = ?**
> 这个数有 20 个 9 呢！谁来试试？

（让学生尝试用 `int` 或 `long long` 存这个数，发现编译报错或结果不对。）

> 哈哈，电脑也"装不下"了！ `int` 最多存大约 21 亿， `long long` 最多存大约 9.2×10^{18}（大约 19 位数字）。如果数字有 100 位、1000 位呢？电脑的"口袋"就不够大了。
> 今天我们要学一个超级厉害的本领——**高精度加法**。学会了它，哪怕数字有一万位，我们也能算得清清楚楚！

4.2 讲解①：核心思路（5′–20′）

（1）生活比喻：竖式加法

> 同学们想一想，你们小学一年级就会做竖式加法了对不对？比如：
> ```
> 3 8 6
> + 4 7 5
> ---
> ```
> 你们是怎么算的？——**从个位开始，一位一位往左加**，满十进一。
> 高精度加法就是让电脑**模拟你做竖式的过程**！

（2）搬家比喻：为什么用数组

> 一个 100 位的大数字，就像一条长长的火车，有 100 节车厢。
> `int` 和 `long long` 就像一个小盒子，只能装 10 节车厢的东西——装不下！
> 那怎么办呢？我们找 100 个小格子（数组），每个格子放一位数字。
>
> | 格子编号 | 0 | 1 | 2 | 3 | ... |
> |---|---|---|---|---|---|
> | 存的数字 | 6 | 8 | 3 | — | ... |
>
> 这就像把火车拆开来，每节车厢单独停在一个停车位上。

（3）为什么倒序存？

> 注意看，上面表格里 386 是**倒着存**的：格子 0 放个位 6，格子 1 放十位 8，格子 2 放百位 3。
> 为什么要倒着呢？因为我们做竖式的时候是**从个位开始加**的——如果个位在 `a[0]`，我们的循环就可以从 `i = 0` 开始，多方便呀！
> 而且，如果最高位产生进位，数字会多一位——倒序存的话，只要在数组后面多加一格就行了，不用把前面所有数字都搬家。

（4）完整流程（板书/PPT）

```
第 1 步：字符串读入两个大数 s1, s2
第 2 步：把字符串倒序转成数组 a[], b[]（字符 '0'~'9' 变成数字 0~9）
第 3 步：从 i=0 开始逐位相加：c[i] = a[i] + b[i] + 进位
第 4 步：处理进位：如果 c[i] >= 10，就 c[i] -= 10，进位 = 1
第 5 步：去掉前导零，倒序输出 c[]
```

4.3 动手①：纸笔模拟（20′–30′）

发一张"高精度竖式模拟表"，让学生手动填写：

> **题目**：用高精度方法计算 **9876 + 2345**
> 请把两个数倒序填入表格，然后逐位相加，写出进位和结果。

位置 i	0（个位）	1（十位）	2（百位）	3（千位）	4（万位）
a[i]	6	7	8	9	0
b[i]	5	4	3	2	0
进位	0	1	1	1	0
c[i]	1	2	2	2	1

> 结果倒序输出：**12221** （验算：9876 + 2345 = 12221）

同学们做完后，请 2–3 位同学上台展示自己的表格，互相对照。

4.4 休息环节小游戏："进位接龙"（50′–55′）

> 规则：老师说两个一位数，同学们要**快速回答**它们的和。如果有进位就大声喊"进一！"
> 比如：老师说"7 和 8"，同学们喊"15！进一！"
> 老师说"3 和 4"，同学们喊"7！"

（既放松了注意力，又强化了进位的概念。）

七、常见错误与应对

错误 1：忘记倒序存储

```
// 错误写法
for (int i = 0; i < len1; i++) {
 a[i] = s1[i] - '0'; // 这样 a[0] 存的是最高位!
}
```

后果：加法从最高位开始加了，进位方向完全反了，结果一塌糊涂。

应对：提醒学生记住口诀——**"个位打头，倒着存"**。可以让学生在代码旁边写注释 `// 倒序存储：a[0] 是个位`。

错误 2：循环结束后忘记处理最后的进位

```
// 忘记了这段!
if (carry > 0) {
 c[len] = carry;
 len++;
}
```

后果：`999 + 1` 会输出 `000` 而不是 `1000`。

应对：告诉学生："循环结束了，但进位可能还在'口袋'里没放出来呢！一定要**最后再检查一次进位**。"

错误 3：字符转数字时忘记 `- '0'`

```
a[i] = s1[len1 - 1 - i]; // 少了 - '0'!
```

后果：`a[i]` 存的是字符的 ASCII 码（比如 48~57），不是 0~9，加出来全是乱七八糟的大数。

应对：提醒学生——字符串里拿出来的是**字符**，不是数字。`'3'` 不等于 `3`！要减去 `'0'` 才能变成数字。

错误 4：数组开得太小

```
int a[100]; // 如果数字有 500 位就越界了!
```

后果：数组越界，程序可能崩溃或输出奇怪的结果。

应对：养成习惯——**数组多开一点**。题目说最多 500 位，我们就开 510，留点余量。

错误 5：没有初始化数组为 0

```
int a[510]; // 没有 = {0}
```

后果：数组里可能有"垃圾值"，导致结果出错。

应对：定义数组时加上 `= {0}`，养成好习惯。或者用 `memset(a, 0, sizeof(a))`。

错误 6：输出前导零

比如计算 `100 + 200 = 300`，如果处理不当可能输出 `0300`。

应对：在本课的标准代码中，因为我们用 `len` 记录了实际位数，倒序输出时从 `len-1` 开始，一般不会有前导零。但如果学生自己改了代码结构，提醒他们注意去掉前导零。特殊情况：两个数都是 0 时，结果应该输出 `0` 而不是空。

九、下节课预告

> 同学们，下节课是**第 24 课：阶段二复习与测试**！
>
> 我们会把第 13 课到第 23 课学过的所有知识点来一次大复习——函数、二维数组、枚举法、模拟法、排序、贪心、还有今天学的高精度。
>
> 然后会有一场**小测试**，检验一下大家这个阶段的学习成果。不用紧张哦，认真复习就好！
>
> **复习建议**：
> 1. 把每节课的例题重新看一遍，确保自己能独立写出来。
> 2. 把课后作业中没有 AC（全部通过）的题目再试一试。
> 3. 特别注意：排序（冒泡/选择/插入/sort）、模拟法、高精度加法——这些是重点！

> **教案备注**：本课是阶段二最后一个新知识点，下节课即为阶段复习与测试。建议在本课结束时给学生足够的鼓励，强调"你们已经掌握了很多厉害的技能"，帮助他们建立信心迎接测试。